SIMD

The Good, the Bad and the Ugly

Viktor Leis

Outline

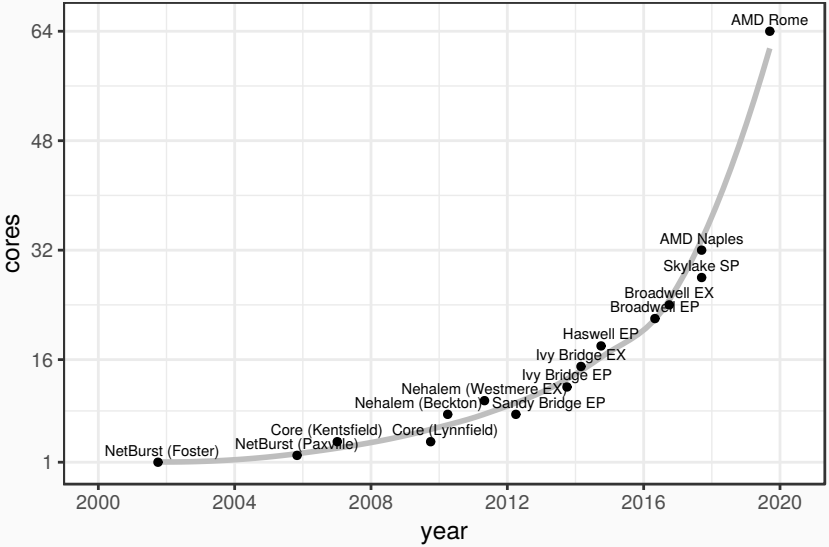
1. Why does SIMD exist?
2. How to use SIMD?
3. When does SIMD help?

Why Does SIMD Exist?

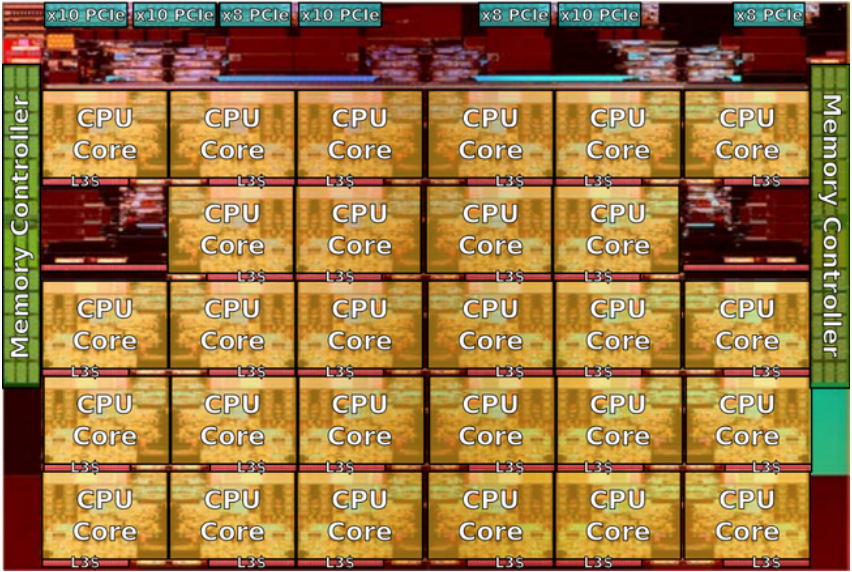
Hardware Trends

- for decades, single-threaded performance doubled every 18-22 months
- during the last decade, single-threaded performance has been stagnating
- clock rates stopped growing due to heat/end of Dennard's scaling
- number of transistors is still growing, used for:
  - large caches
  - many cores (thread parallelism)
  - SIMD (data parallelism)

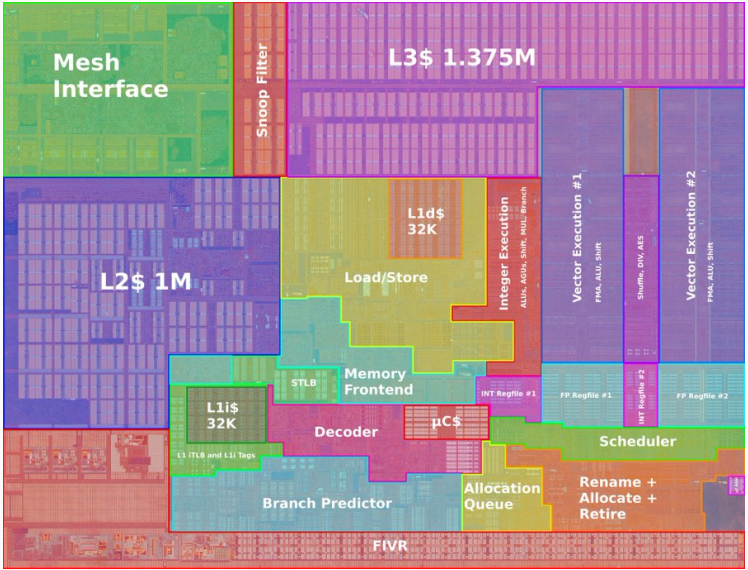
Number of Cores



Skylake CPU


x10 PCIe
x10 PCIe
x8 PCIe
x10 PCIe
x8 PCIe
x10 PCIe
x8 PCIe
Memory Controller
Memory Controller
CPU Core
L3$

Skylake Core

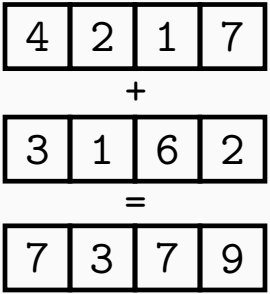


x86 SIMD History

- 1997: MMX 64-bit (Pentium 1)
- 1999: SSE 128-bit (Pentium 3)
- 2011: AVX 256-bit float (Sandy Bridge)
- 2013: AVX2 256-bit int (Haswell)
- 2017: AVX-512 512 bit (Skylake Server)
- instructions usually introduced by Intel
- AMD typically followed quickly (except for AVX-512)

Single Instruction Multiple Data (SIMD)

- data parallelism exposed by the CPU's instruction set
- CPU register holds multiple fixed-size values (e.g., 4×32-bit)
- instructions (e.g., addition) are performed on two such registers (e.g., executing 4 additions in one instruction)



Registers

- SSE: 16×128 bit (XMM0...XMM15)
- AVX: 16×256 bit (YMM0...YMM15)
- AVX-512: 32×512 bit (ZMM0...ZMM31)
- XMM0 is lower half of YMM0
- YMM0 is lower half of ZMM0

256-Bit Register May Contain

- 32×8-bit integers
- 16×16-bit integers
- 8×32-bit integers
- 4×64-bit integers
- 8×32-bit floats
- 4×64-bit floats

Theoretical Speedup for Integer Code

- most Intel and AMD CPUs can execute 4 integer and 2 vector instructions per cycle
- maximum SIMD speedup depends on:
  - SIMD vector width (128-, 256-, or 512-bit)
  - data type (8-, 16-, 32-, 64-bit), integer/float
- examples for maximum speedup:
  - AVX512, 8-bit: 32×
  - AVX2, 16-bit: 8×
  - SSE, 32-bit: 2×

How to Use SIMD?

Auto-Vectorization

- compilers try to transform serial code into SIMD code
- this does not work for complicated code and is quite unpredictable
- sometimes compilers are quite clever
- requires specifying compilation target, otherwise only SSE2 is available on x86-64

No Auto-Vectorization

Compiler Explorer

https://godbolt.org

C++ source #1

```cpp
#include <cstdint>
#include <cstddef>

static const int n = 16*1024*1024;

int v[n];

int f() {
    int s = 0;
    #pragma clang loop vectorize(disable)
    for (int i=0; i<n; i++) {
        s += v[i];
    }
    return s;
}
```

x86-64 clang 9.0.0 (Editor #1, Compiler #1) C++

x86-64 clang 9.0.0 -O3 -std=c++14

```asm
f():                                    # @f()
        xor     ecx, ecx
        xor     eax, eax
.LBB0_1:                                # =>This Inner Loop
        add     eax, dword ptr [rcx + v]
        add     eax, dword ptr [rcx + v+4]
        add     eax, dword ptr [rcx + v+8]
        add     eax, dword ptr [rcx + v+12]
        add     eax, dword ptr [rcx + v+16]
        add     eax, dword ptr [rcx + v+20]
        add     eax, dword ptr [rcx + v+24]
        add     eax, dword ptr [rcx + v+28]
        add     rcx, 32
        cmp     rcx, 67108864
        jne     .LBB0_1
        ret
v:
        .zero   67108864
```

Auto-Vectorization: SSE

https://godbolt.org

COMPILER EXPLORER

Add... More Sponsors No bugs Solid Ping Share Other Policies

C++ source #1

C++

```cpp
#include <cstdint>

int f(int* v) {
    int s = 0;
    for (int i=0; i<1024*1024; i++) {
        s += v[i];
    }
    return s;
}
```

x86-64 clang 9.0.0 (Editor #1, Compiler #1) C++

x86-64 clang 9.0.0 -O3 -std=c++14

Output... Filter... Libraries + Add new... Add tool...

```asm
f(int*):                                # @f(int*)
        pxor    xmm0, xmm0
        xor     eax, eax
        pxor    xmm1, xmm1
.LBB0_1:                                # =>This Inner Loop Header: De
        movdqu  xmm2, xmmword ptr [rdi + 4*rax]
        paddd   xmm2, xmm0
        movdqu  xmm0, xmmword ptr [rdi + 4*rax + 16]
        paddd   xmm0, xmm1
        movdqu  xmm1, xmmword ptr [rdi + 4*rax + 32]
        movdqu  xmm3, xmmword ptr [rdi + 4*rax + 48]
        movdqu  xmm4, xmmword ptr [rdi + 4*rax + 64]
        paddd   xmm4, xmm1
        paddd   xmm4, xmm2
        movdqu  xmm2, xmmword ptr [rdi + 4*rax + 80]
        paddd   xmm2, xmm3
        paddd   xmm2, xmm0
        movdqu  xmm0, xmmword ptr [rdi + 4*rax + 96]
        paddd   xmm0, xmm4
        movdqu  xmm1, xmmword ptr [rdi + 4*rax + 112]
        paddd   xmm1, xmm2
        add     rax, 32
        cmp     rax, 1048576
        jne     .LBB0_1
        paddd   xmm1, xmm0
        pshufd  xmm0, xmm1, 78          # xmm0 = xmm1[2,3,0,1]
        paddd   xmm0, xmm1
        pshufd  xmm1, xmm0, 229         # xmm1 = xmm0[1,1,2,3]
        paddd   xmm1, xmm0
        movd    eax, xmm1
        ret
```

Auto-Vectorization: AVX2

https://godbolt.org

COMPILER EXPLORER

Add... More Sponsors Share Other Policies

C++ source #1

C++

```cpp
#include <cstdint>

int f(int* v) {
    int s = 0;
    for (int i=0; i<1024*1024; i++) {
        s += v[i];
    }
    return s;
}
```

x86-64 clang 9.0.0 (Editor #1, Compiler #1) C++

x86-64 clang 9.0.0

-O3 -std=c++14 -march=haswell

Output... Filter... Libraries + Add new... Add tool...

```asm
f(int*):                                # @f(int*)
        vpxor   xmm0, xmm0, xmm0
        xor     eax, eax
        vpxor   xmm1, xmm1, xmm1
        vpxor   xmm2, xmm2, xmm2
        vpxor   xmm3, xmm3, xmm3
.LBB0_1:                                # =>This Inner Loop Header: De
        vpaddd  ymm0, ymm0, ymmword ptr [rdi + 4*rax]
        vpaddd  ymm1, ymm1, ymmword ptr [rdi + 4*rax + 32]
        vpaddd  ymm2, ymm2, ymmword ptr [rdi + 4*rax + 64]
        vpaddd  ymm3, ymm3, ymmword ptr [rdi + 4*rax + 96]
        vpaddd  ymm0, ymm0, ymmword ptr [rdi + 4*rax + 128]
        vpaddd  ymm1, ymm1, ymmword ptr [rdi + 4*rax + 160]
        vpaddd  ymm2, ymm2, ymmword ptr [rdi + 4*rax + 192]
        vpaddd  ymm3, ymm3, ymmword ptr [rdi + 4*rax + 224]
        vpaddd  ymm0, ymm0, ymmword ptr [rdi + 4*rax + 256]
        vpaddd  ymm1, ymm1, ymmword ptr [rdi + 4*rax + 288]
        vpaddd  ymm2, ymm2, ymmword ptr [rdi + 4*rax + 320]
        vpaddd  ymm3, ymm3, ymmword ptr [rdi + 4*rax + 352]
        vpaddd  ymm0, ymm0, ymmword ptr [rdi + 4*rax + 384]
        vpaddd  ymm1, ymm1, ymmword ptr [rdi + 4*rax + 416]
        vpaddd  ymm2, ymm2, ymmword ptr [rdi + 4*rax + 448]
        vpaddd  ymm3, ymm3, ymmword ptr [rdi + 4*rax + 480]
        sub     rax, -128
        cmp     rax, 1048576
        jne     .LBB0_1
        vpaddd  ymm0, ymm1, ymm0
        vpaddd  ymm0, ymm2, ymm0
        vpaddd  ymm0, ymm3, ymm0
        vextracti128    xmm1, ymm0, 1
        vpaddd  xmm0, xmm0, xmm1
        vpshufd xmm1, xmm0, 78          # xmm1 = xmm0[2,3,0,1]
        vpaddd  xmm0, xmm0, xmm1
        vpshufd xmm1, xmm0, 229         # xmm1 = xmm0[1,1,2,3]
        vpaddd  xmm0, xmm0, xmm1
        vmovd   eax, xmm0
        vzeroupper
        ret
```

Auto-Vectorization: AVX-512

https://godbolt.org

COMPILER EXPLORER

Add... More Sponsors Share Other Policies

C++ source #1

C++

```cpp
#include <cstdint>

int f(int* v) {
    int s = 0;
    for (int i=0; i<1024*1024; i++) {
        s += v[i];
    }
    return s;
}
```

x86-64 clang 9.0.0 (Editor #1, Compiler #1) C++

x86-64 clang 9.0.0

-O3 -std=c++14 -march=skylake-avx512

Output... Filter... Libraries + Add new... Add tool...

```asm
f(int*):                                # @f(int*)
        vpxor   xmm0, xmm0, xmm0
        xor     eax, eax
        vpxor   xmm1, xmm1, xmm1
        vpxor   xmm2, xmm2, xmm2
        vpxor   xmm3, xmm3, xmm3
.LBB0_1:                                # =>This Inner Loop Header: De
        vpaddd  zmm0, zmm0, zmmword ptr [rdi + 4*rax]
        vpaddd  zmm1, zmm1, zmmword ptr [rdi + 4*rax + 64]
        vpaddd  zmm2, zmm2, zmmword ptr [rdi + 4*rax + 128]
        vpaddd  zmm3, zmm3, zmmword ptr [rdi + 4*rax + 192]
        vpaddd  zmm0, zmm0, zmmword ptr [rdi + 4*rax + 256]
        vpaddd  zmm1, zmm1, zmmword ptr [rdi + 4*rax + 320]
        vpaddd  zmm2, zmm2, zmmword ptr [rdi + 4*rax + 384]
        vpaddd  zmm3, zmm3, zmmword ptr [rdi + 4*rax + 448]
        vpaddd  zmm0, zmm0, zmmword ptr [rdi + 4*rax + 512]
        vpaddd  zmm1, zmm1, zmmword ptr [rdi + 4*rax + 576]
        vpaddd  zmm2, zmm2, zmmword ptr [rdi + 4*rax + 640]
        vpaddd  zmm3, zmm3, zmmword ptr [rdi + 4*rax + 704]
        vpaddd  zmm0, zmm0, zmmword ptr [rdi + 4*rax + 768]
        vpaddd  zmm1, zmm1, zmmword ptr [rdi + 4*rax + 832]
        vpaddd  zmm2, zmm2, zmmword ptr [rdi + 4*rax + 896]
        vpaddd  zmm3, zmm3, zmmword ptr [rdi + 4*rax + 960]
        add     rax, 256
        cmp     rax, 1048576
        jne     .LBB0_1
        vpaddd  zmm0, zmm1, zmm0
        vpaddd  zmm0, zmm2, zmm0
        vpaddd  zmm0, zmm3, zmm0
        vextracti64x4   ymm1, zmm0, 1
        vpaddd  zmm0, zmm0, zmm1
        vextracti128    xmm1, ymm0, 1
        vpaddd  xmm0, xmm0, xmm1
        vpshufd xmm1, xmm0, 78          # xmm1 = xmm0[2,3,0,1]
        vpaddd  xmm0, xmm0, xmm1
        vpshufd xmm1, xmm0, 229         # xmm1 = xmm0[1,1,2,3]
        vpaddd  xmm0, xmm0, xmm1
        vmovd   eax, xmm0
        vzeroupper
        ret
```

Code Performance Profile

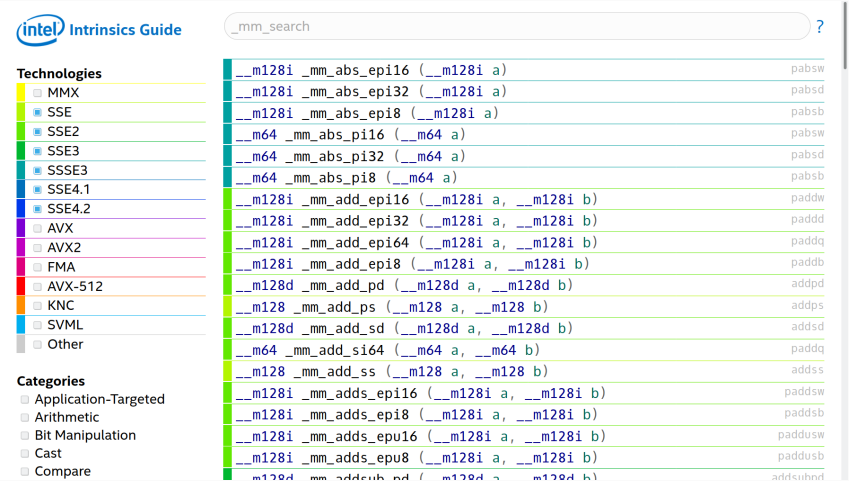
- use perf event framework to get CPU metrics (on Linux)
- sum experiment on Skylake X (4 billion iterations, data in L1 cache):

```
   , time, cyc,  inst,  L1m, LLCm, brmiss,  IPC,  CPU,  GHz
sca, 1.00, 1.00, 1.25, 0.00, 0.00,   0.00, 1.25, 1.00, 4.29
SSE, 0.13, 0.13, 0.32, 0.00, 0.00,   0.00, 2.37, 1.00, 4.29
AVX, 0.06, 0.06, 0.15, 0.00, 0.00,   0.00, 2.27, 1.00, 4.30
512, 0.03, 0.03, 0.08, 0.00, 0.00,   0.00, 2.24, 1.00, 4.28
```

Intrinsics

- C/C++ interface to SIMD instructions without having to write assembly code
- SIMD registers are represented as special data types:
  - integers (all widths): `__m128i`/`__m256i`/`__m512i`
  - floats (32-bit): `__m128`/`__m256`/`__m512`
  - floats (64-bit): `__m128d`/`__m256d`/`__m512d`
- operations look like C functions, e.g., add 16 32-bit integers:
  `__m512i _mm512_add_epi32(__m512i a, __m512i b)`
- C compiler performs register allocation
- a SIMD interface is also available in Java, but uncommon

Intel Intrinsics Guide



https://software.intel.com/sites/landingpage/IntrinsicsGuide/

Intrinsics Example

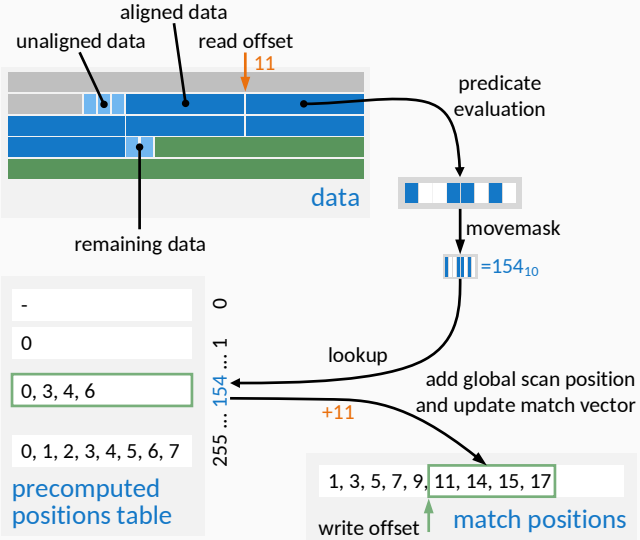
```
alignas(64) int in[1024];
void simpleMultiplication() {
   __m512i three =  _mm512_set1_epi32(3);
   for (int i=0; i<1024; i+=16) {
      __m512i x = _mm512_load_si512(in + i);
      __m512i y = _mm512_mullo_epi32(x, three);
      _mm512_store_epi32(in + i, y); }}
//-------------------------------------------------------------

        xor       eax, eax
        vmovups   zmm0, ZMMWORD PTR CONST.0[rip]
.LOOP:
        vpmulld   zmm1, zmm0, ZMMWORD PTR [in+rax*4]
        vmovups   ZMMWORD PTR [in+rax*4], zmm1
        add       rax, 16
        cmp       rax, 1024
        jl        .LOOP
        ret
.CONST.0: .long   0x00000003,0x00000003,...
```

Basic Instructions

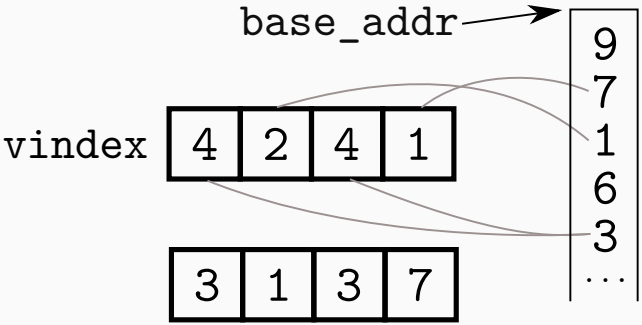
- load/store: load/store vector from/to contiguous memory location
- arithmetic: addition/subtraction, multiplication, min/max, absolute value
- logical: and, or, not, xor, rotate, shift, convert
- comparisons: less, equal
- no integer division, no overflow detection, mostly signed integers
- instruction set not quite orthogonal (missing instruction variants)

Hyper Data Blocks Selection Example (Lang et al., SIGMOD 2016)

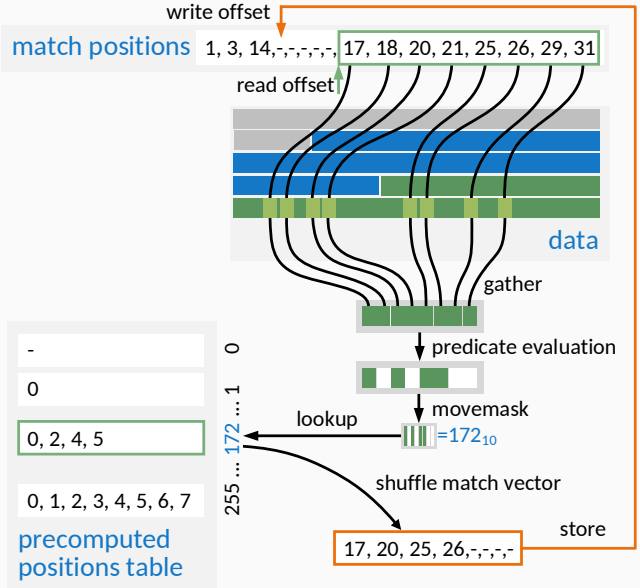


Gather

- perform loads from multiple locations, combine results in one register
- unfortunately quite slow (2 items per cycles)



Hyper Data Blocks Gather Example (Lang et al., SIGMOD 2016)



Permute

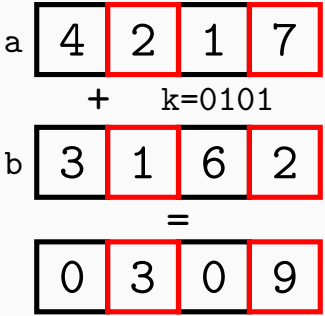
- permute (also called shuffle) `a` using the corresponding index in `idx`:
  `__m512i _mm512_permutexvar_epi32 (__m512i idx, __m512i a)`
- a bit of a misnomer, is not just shuffle or permute but can replicate elements
- very powerful, can, e.g., be used to implement small, in-register lookup tables

a	4	2	1	7
idx	1	0	3	0
	2	4	7	4

AVX512-Only: Masking

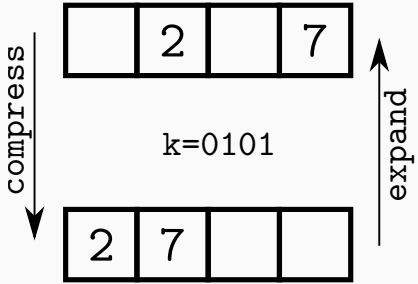
- selectively ignore some of the SIMD lanes using a bitmap
- allows expressing complex control flow
- almost all operations, including comparisons, support masking
- add elements, but set those not selected by mask to zero:

```
__m512i _mm512_maskz_add_epi32 (__mmask16 k, __m512i a, __m512i b)
```



AVX512-Only: Compress and Expand

- compress: `__m512i _mm512_maskz_compress_epi32(__mmask16 k, __m512i a)`
- expand: `__m512i _mm512_maskz_expand_epi32(__mmask16 k, __m512i a)`



Vector Classes

- reading/writing intrinsic code is cumbersome
- to make the code more readable, wrap register in a Vector class:

```
struct Vec8u {
   union {
      __m512i reg;
      uint64_t entry[8];
   };

   // constructors
   Vec8u(uint64_t x) { reg = _mm512_set1_epi64(x); };
   Vec8u(void* p) { reg = _mm512_loadu_si512(p); };
   Vec8u(__m512i x) { reg = x; };
   Vec8u(uint64_t x0, uint64_t x1, uint64_t x2, uint64_t x3, uint64_t x4, uint64_t x5,
         uint64_t x6, uint64_t x7) { reg = _mm512_set_epi64(x0,x1,x2,x3,x4,x5,x6,x7); };

   // implicit conversion to register
   operator __m512i() { return reg; }
```

Vector Classes (2)

```
  friend std::ostream& operator<< (std::ostream& stream, const Vec8u& v) {
    for (auto& e : v.entry) stream << e << " "; return stream; }
};

inline Vec8u operator+ (const Vec8u& a, const Vec8u& b) {
  return _mm512_add_epi64(a.reg, b.reg); }
inline Vec8u operator- (const Vec8u& a, const Vec8u& b) {
  return _mm512_sub_epi64(a.reg, b.reg); }
inline Vec8u operator* (const Vec8u& a, const Vec8u& b) {
  return _mm512_mullo_epi64(a.reg, b.reg); }
inline Vec8u operator^ (const Vec8u& a, const Vec8u& b) {
  return _mm512_xor_epi64(a.reg, b.reg); }
inline Vec8u operator>> (const Vec8u& a, const unsigned shift) {
  return _mm512_srli_epi64(a.reg, shift); }
inline Vec8u operator<< (const Vec8u& a, const unsigned shift) {
  return _mm512_slli_epi64(a.reg, shift); }
inline Vec8u operator& (const Vec8u& a, const Vec8u& b) {
  return _mm512_and_epi64(a.reg, b.reg); }
```

Vector Classes (3)

```
Vec8u murmurHash(uint64_t* scalarKeys) {
   const int r = 47;
   Vec8u m = 0xc6a4a7935bd1e995ull;
   Vec8u k = scalarKeys;
   Vec8u h = 0x8445d61a4e774912ull ^ (8*m);
   k = k * m;
   k = k ^ (k >> r);
   k = k * m;
   h = h ^ k;
   h = h * m;
   h = h ^ (h >> r);
   h = h * m;
   h = h ^ (h >> r);
   return h;
}
```

Cross-Lane Operations, Reductions

- sometimes one would like to perform operations *across* lanes
- except for some exceptions (e.g., permute), this cannot be done efficiently
- horizontal operations violate the SIMD paradigm
- for example, there is an intrinsic for adding register values:
  `int _mm512_reduce_add_epi32 (__m512i a)`
- but (if supported by the compiler), it is translated to multiple SIMD instructions (by the compiler) and is therefore less efficient than `_mm512_add_epi32`

SSE4.2

- SSE4.2 provides fast CRC error-correction codes (8 bytes/cycle, can be used for hashing too)
- SSE4.2 also provides fast string comparisons/search
- compares two 16-byte chunks, e.g., to find out whether (and where) a string contains a set of characters
- very flexible: len/zero-terminated strings, any/substring search, bitmask/index result, etc.
- tool for finding settings: `http://halobates.de/pcmpstr-js/pcmp.html`

When Does SIMD Help?

Which Version To Target?

- **all** x86-64 CPUs support at least SSE2
- SSE3 and SSE4.2 are almost universal
- AVX2 fairly widespread
- AVX-512 quite rare: Skylake and Cascade Lake Servers only, no AMD support

- lowest common denominator: SSE2
- compile and ship multiple binaries, on startup call the best one
- multiple versions per function, dynamically call the best one

When Does SIMD Help?

- computation-bound, throughput-oriented workloads
- small data types
- working on many items
- floating point computation
- examples: decompression, working on compressed data, table scan code, matrix multiplication

When Does SIMD Not Help?

- code too complex/large to vectorize
- memory bandwidth bound
- (memory-)latency bound
- low IPC-workloads
- examples: hash joins, generated tuple-at-a-time code in Hyper

Other Caveats

- SIMD code may result in lower clock frequencies
- I have not really seen this for integer SIMD code
- switching from legacy xmm code and avx code, may cause 70-cycle delay
  - to avoid this Intel recommends executing vzeroupper instruction after AVX code
  - this clears upper bits of ymm/zmm registers and is much cheaper

Links/Further Reading

- Intel Intrinsics Guide: `https://software.intel.com/sites/landingpage/IntrinsicsGuide/`
- Compiler Explorer: `https://godbolt.org/`
- Compiler Explorer Source: `https://github.com/compiler-explorer/compiler-explorer`
- What is the status of VZEROUPPER use? `https://software.intel.com/en-us/forums/intel-isa-extensions/topic/704023`
- Agner Fog's optimization guides: `https://www.agner.org/optimize/`
- header-only perf event library: `https://github.com/viktorleis/perfevent/`